```
def ultimo_dig(n):
    return n % 10
def resto(n):
    return n // 10
def digitos(n):
    if n < 10:
        return 1
    else:
        return 1 + digitos(resto(n))
def suma_digitos(n):
    if n < 10:
        return n
    else:
        u = ultimo_dig(n)
        r = resto(n)
        return u + suma_digitos(r)
def invertir_numero(n):
    # asume n >= 0
    if n < 10:
        return n
    else:
        u = n % 10          # último dígito
        r = n // 10         # resto sin el último
        k = digitod(r)      # cuántos dígitos tiene r
        return u * (10 ** k) + invertir_numero(r)
```

```
def max_digito(n):
    # sin usar max()
    if n < 10:
        return n
    else:
        r = max_digito(resto(n))
        u = ultimo_dig(n)
        if u >= r:
            return u
        else:
            return r
def suma_potencias(n):
    if n < 10:
        return digito_pot(n)
    else:
        u = ultimo_dig(n)
        r = resto(n)
        return digito_pot(u) + suma_potencias(r)
def primero_munchausen(x, y):
    if x > y:
        return -1
    else:
        if es_munchausen(x):
            return x
        else:
            return primero_munchausen(x + 1, y)

def casi_igual(a, b, eps=1e-9):
    if abs(a - b) < eps:
        return True
    else:
        return False
```

```
def sumaProductos(x,y):
    if x==0 or y==0:
        return 0
    else:
        return (x%10)*(y%10) +
sumaProductos(x//10,y//10)
def contiene(numero,digito):
    if numero<10:
        if digito==numero:
            return True
        else:
            return False
    if numero%10 == digito:
        return True
    else:
        return contiene(numero/10,digito)
def mover(casilla):
    import random
    dado1=random.randint(1,6)
    dado2=random.randint(1,6)
    suma=dado1+dado2+casilla
    if suma>63:
        nuevaCasilla=63-(suma-63)
    else:
        nuevaCasilla=suma
    if nuevaCasilla==63:
        return 63
    elif nuevaCasilla%9==0:
        return nuevaCasilla + 9
    else:
        return nuevaCasilla
```

```
#calcula número de dígitos de n que superan a d
def mayorQue(n,d):
    if n < 10:
        if n > d:
            return 1
        else:
            return 0
    else:
        if n%10 > d:
            return mayorQue(n//10,d) + 1
        else:
            return mayorQue(n//10,d)
```

Modelo de función recursiva

def función (parametros):
CasoBase:
return resultadoBase
CasoRecursivo:
return llamadoRecursivo
	Ejemplo:
	def potencia(x,y):
	if y == 0:
	return 1
	return x * potencia(x, y-1)

Receta de diseño:

1) **Contrato** —-> **#areaRectangulo: num, num -> num**
2) **Descripción** —-> **#calcula el area de un rectángulo dados sus lados**
3) **Ejemplo** —-> **#ejemplo: areaRectangulo(3,4) entrega 12**
4) **Firma** —-> **def areaRectangulo(largo,ancho):**
5) **Definición** —-> **instrucciones...**
return largo * ancho
6) **Testing** —-> **#Test**
assert areaRectangulo(3,4) == 12

// → corta números a la derecha. ej: 1234//10 = 123

% → extrae números a la derecha ej: 1234%10 = 4